

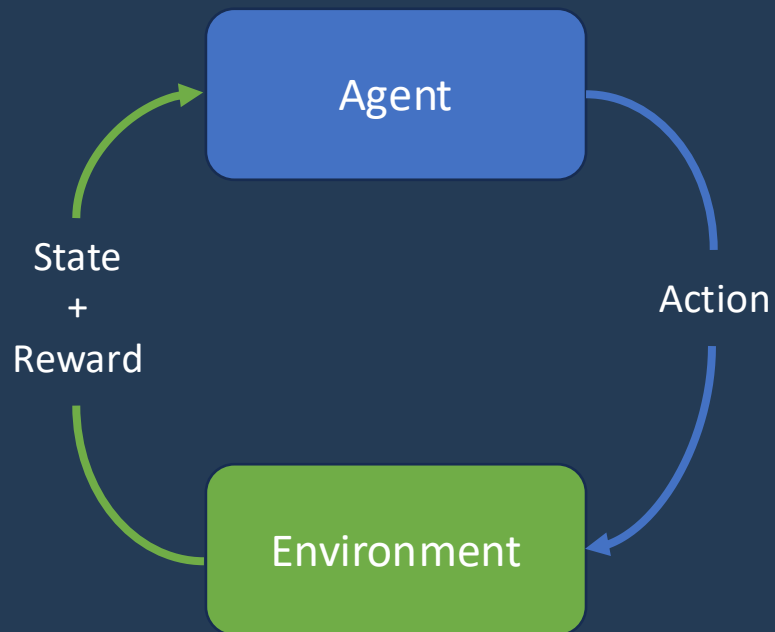
MuZero Knockoff

RL - MCTS – NN - 2048

IT3105 AI Programming - Spring 2026

Jørgen Hauan Larsen, Liam Robert Shepard, Torstein Hauge Fossnes

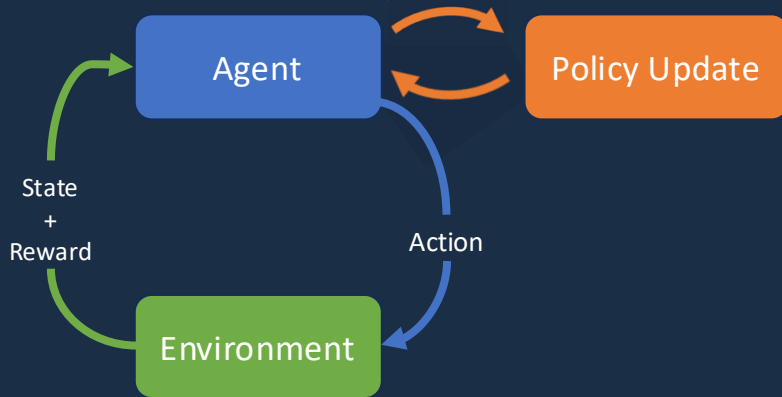
Reinforcement Learning



Model-Free vs Model-Based RL

Model-Free

- No internal understanding of the environment
- Learns state to action function directly.
- Sample inefficient (needs a lot of experience to learn)



Model-Based

- Learns or is given a model of the environment.
- Use the world model to simulate the consequences of actions
- The results inform their next action

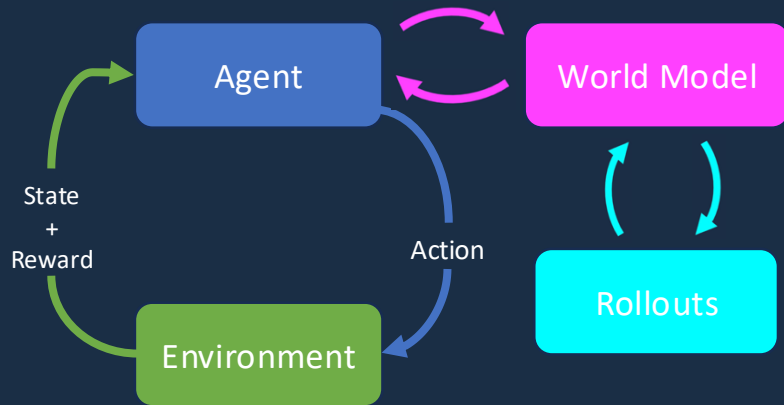
Model-Free vs Model-Based RL

Model-Free

- No internal understanding of the environment
- Learns state to action function directly.
- Sample inefficient (needs a lot of experience to learn)

Model-Based

- Learns or is given a model of the environment.
- Use the world model to simulate the consequences of actions
- The results inform their next action



Monte Carlo Tree Search

01 Select

A tree policy is used to find the most relevant node in the tree

02 Expand

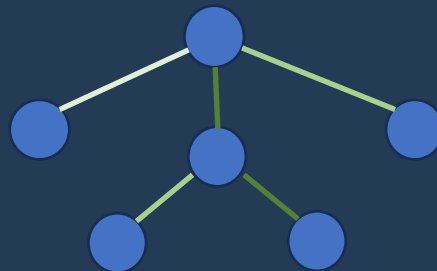
Add child nodes to the leaf representing possible next states

03 Rollout

Play out actions from a child to estimate position value

04 Backpropagate

Uses the rewards from the rollout to update all nodes up to the root



After many iterations, visit counts at the root give a strong action distribution.

Monte Carlo Tree Search

01 Select

A tree policy is used to find the most relevant node in the tree

02 Expand

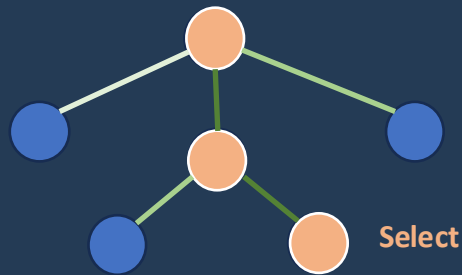
Add child nodes to the leaf representing possible next states

03 Rollout

Play out actions from a child to estimate position value

04 Backpropagate

Uses the rewards from the rollout to update all nodes up to the root



After many iterations, visit counts at the root give a strong action distribution.

Monte Carlo Tree Search

01 Select

A tree policy is used to find the most relevant node in the tree

02 Expand

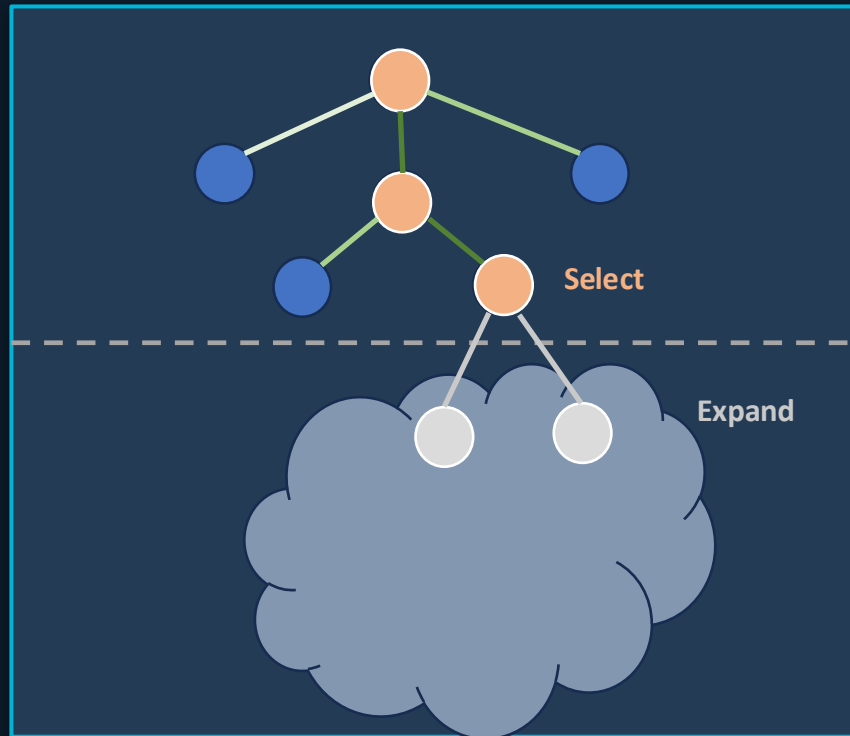
Add child nodes to the leaf representing possible next states

03 Rollout

Play out actions from a child to estimate position value

04 Backpropagate

Uses the rewards from the rollout to update all nodes up to the root



After many iterations, visit counts at the root give a strong action distribution.

Monte Carlo Tree Search

01 Select

A tree policy is used to find the most relevant node in the tree

02 Expand

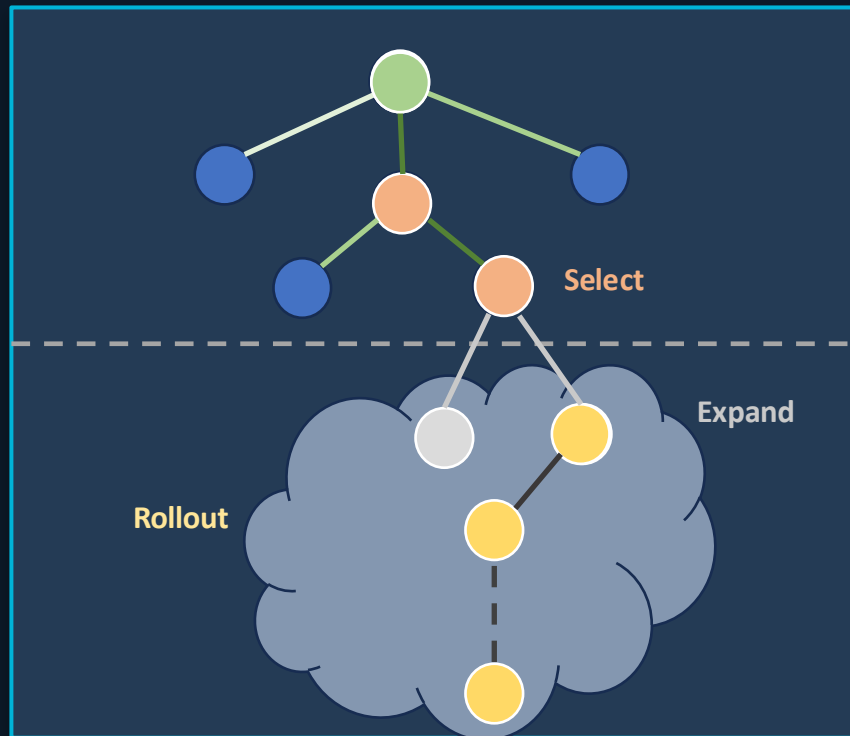
Add child nodes to the leaf representing possible next states

03 Rollout

Play out actions from a child to estimate position value

04 Backpropagate

Uses the rewards from the rollout to update all nodes up to the root



After many iterations, visit counts at the root give a strong action distribution.

Monte Carlo Tree Search

01 Select

A tree policy is used to find the most relevant node in the tree

02 Expand

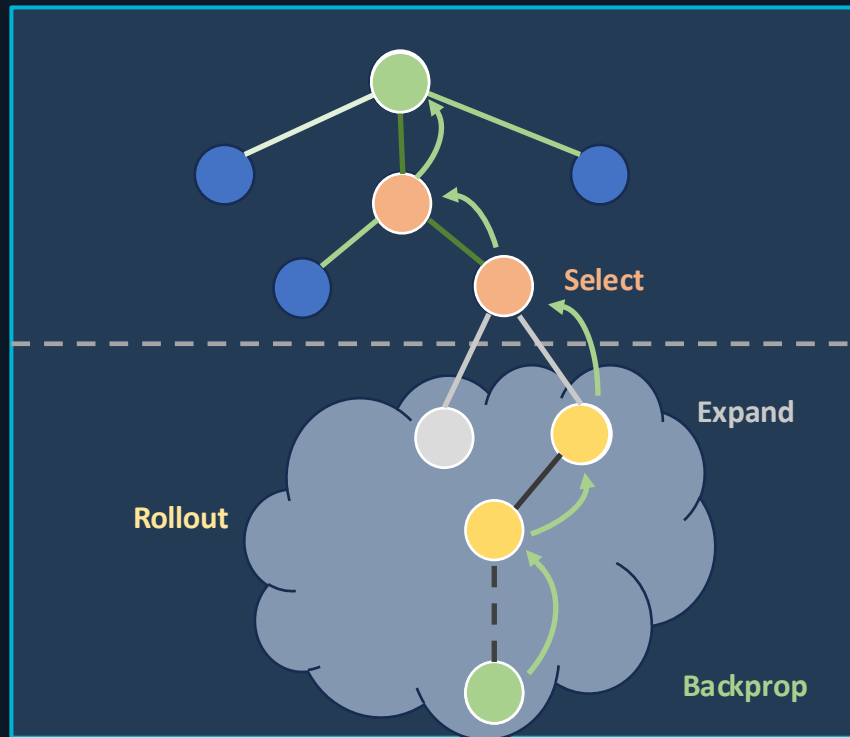
Add child nodes to the leaf representing possible next states

03 Rollout

Play out actions from a child to estimate position value

04 Backpropagate

Uses the rewards from the rollout to update all nodes up to the root



After many iterations, visit counts at the root give a strong action distribution.

MuZero: u-MCTS & The Three Networks (Ψ)

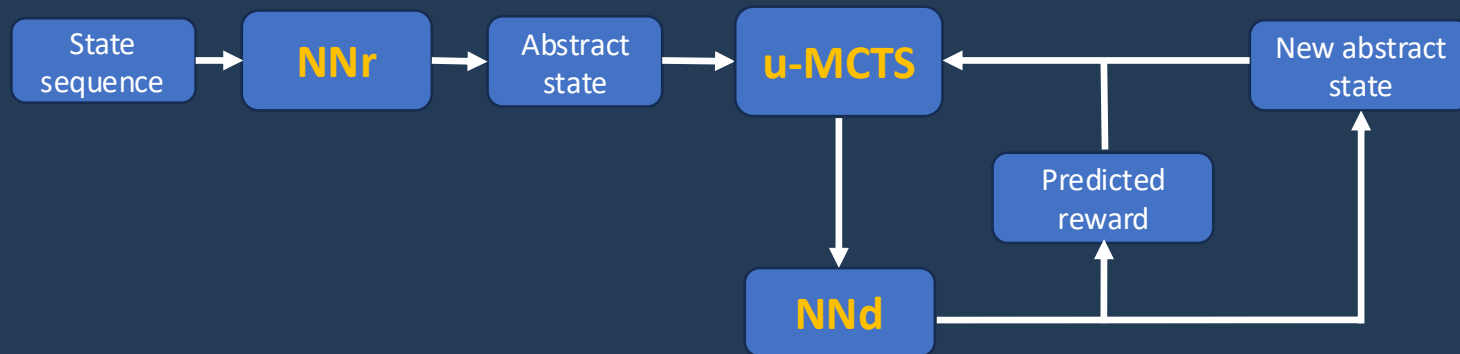


u-MCTS

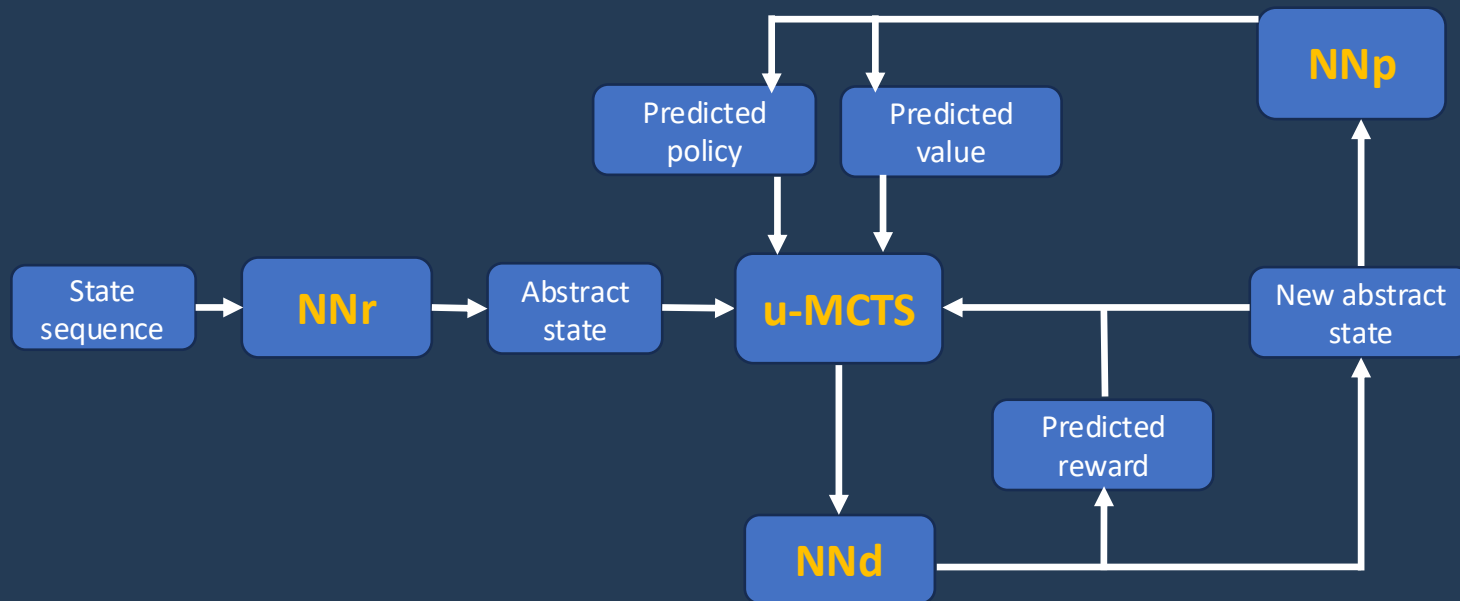
MuZero: u-MCTS & The Three Networks (Ψ)



MuZero: u-MCTS & The Three Networks (Ψ)

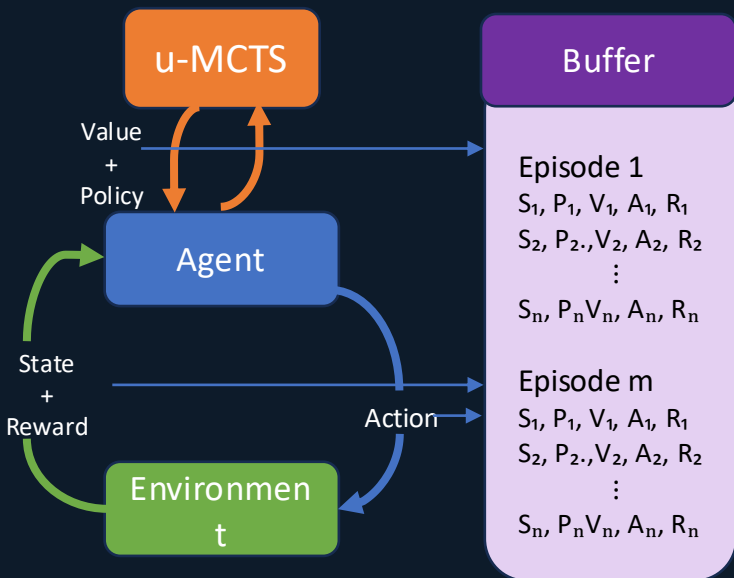


MuZero: u-MCTS & The Three Networks (Ψ)

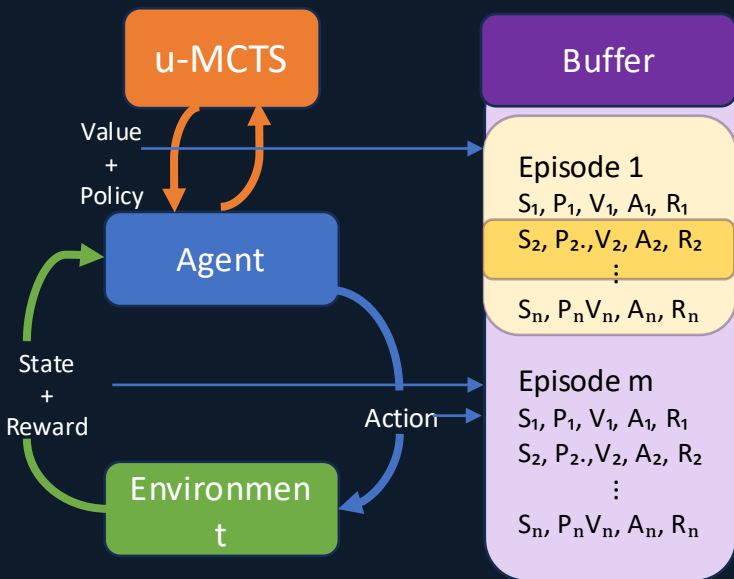


MuZero: Data generation and BPTT

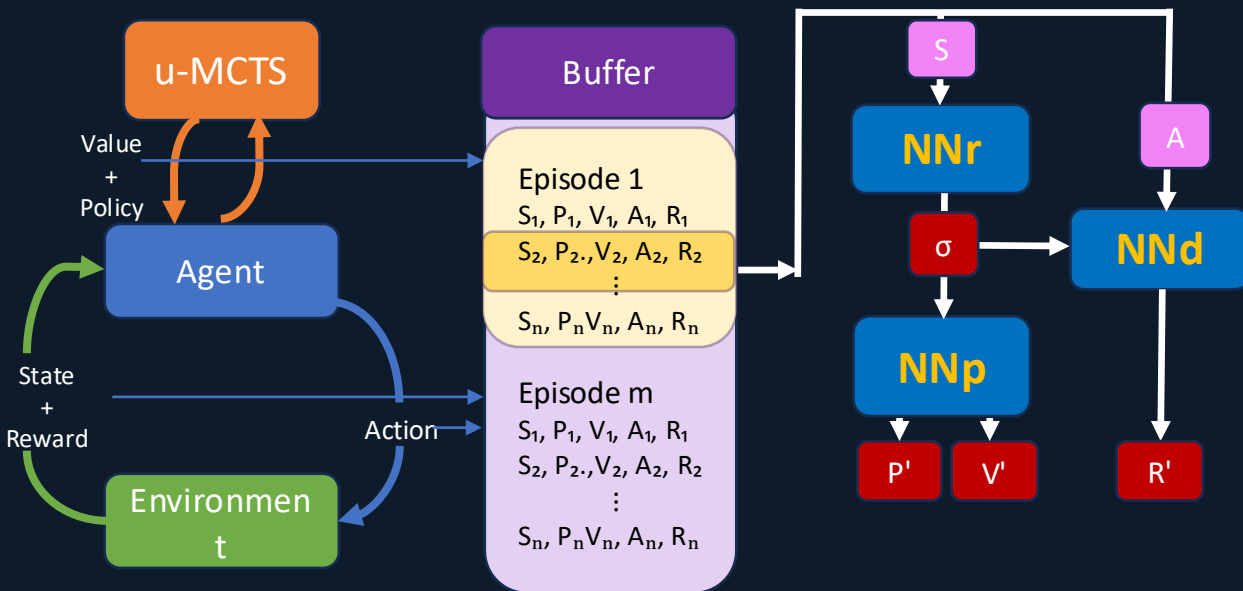
MuZero: Data generation and BPTT



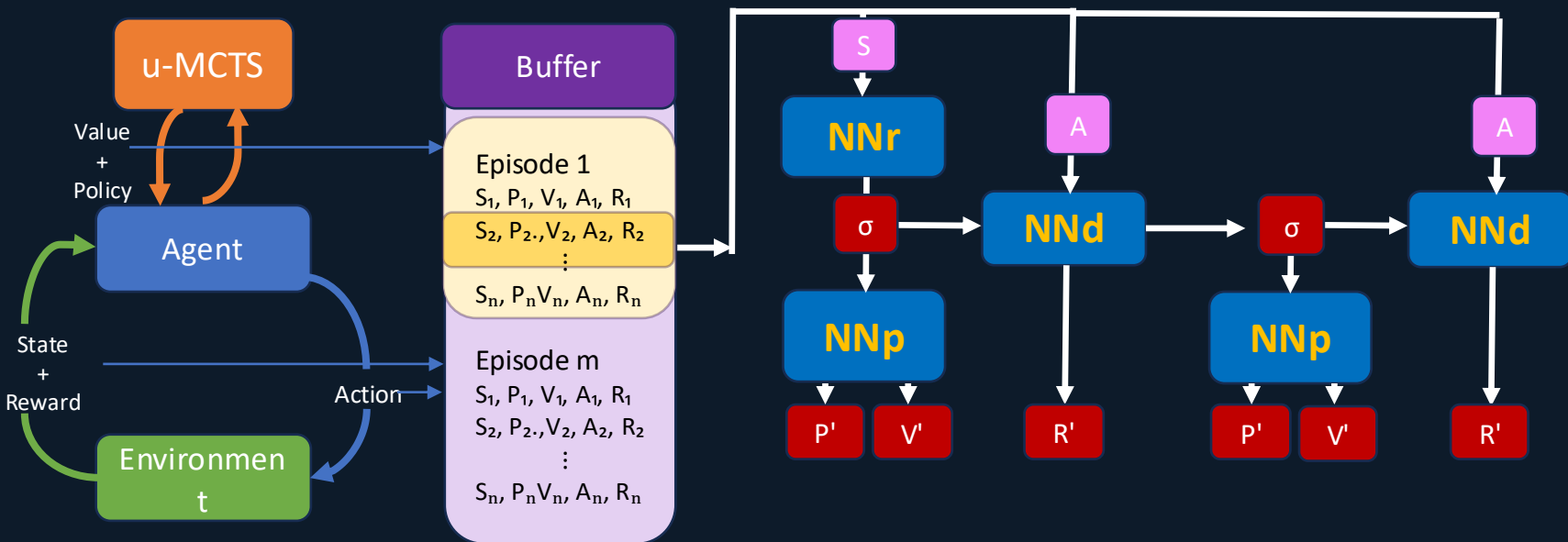
MuZero: Data generation and BPTT



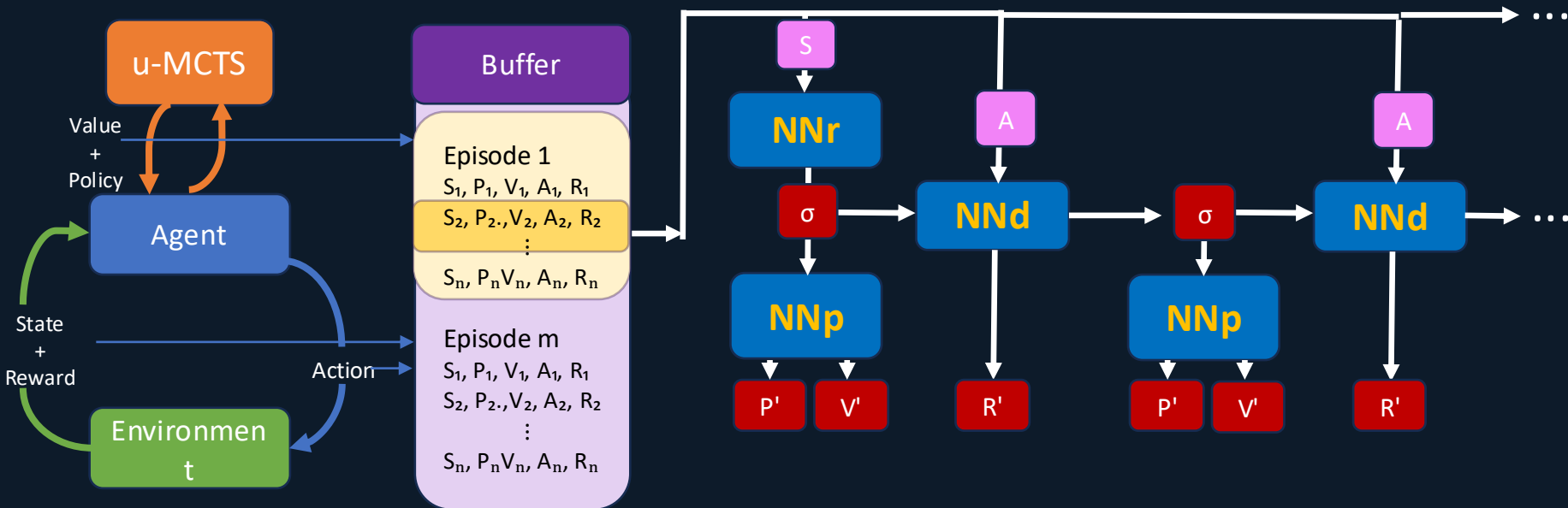
MuZero: Data generation and BPTT

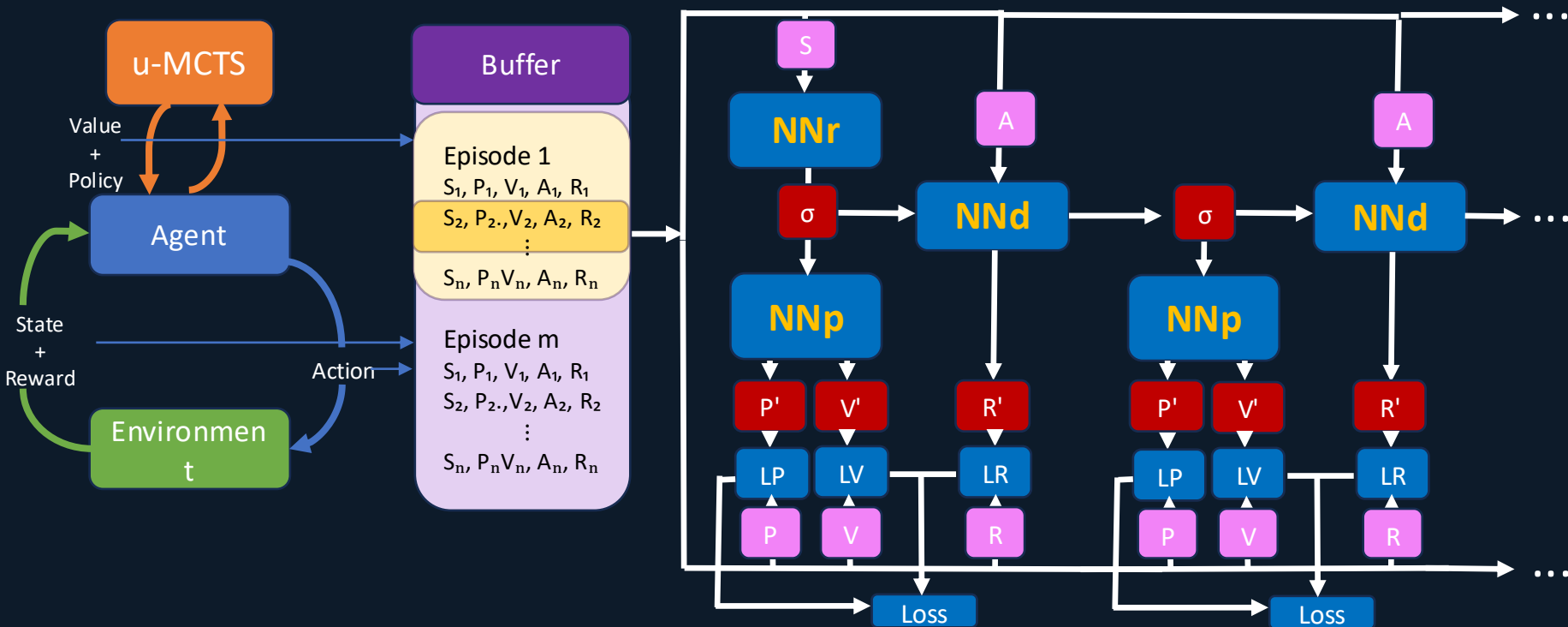


MuZero: Data generation and BPTT



MuZero: Data generation and BPTT





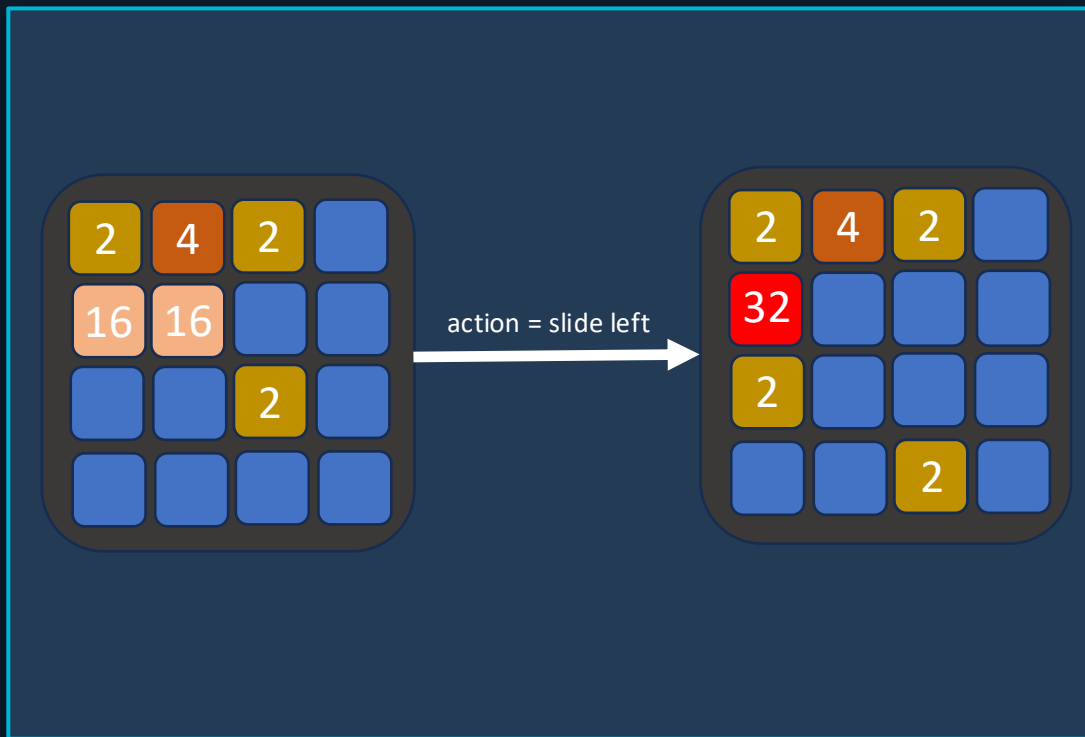
The Game: 2048

- Single-player sliding tile puzzle on a 4×4 grid
- Equal tiles merge and double
- New tile appears randomly after each move
- Goal: reach the 2048 tile without getting stuck



The Game: 2048

- Single-player sliding tile puzzle on a 4×4 grid
- Equal tiles merge and double
- New tile appears randomly after each move
- Goal: reach the 2048 tile without getting stuck

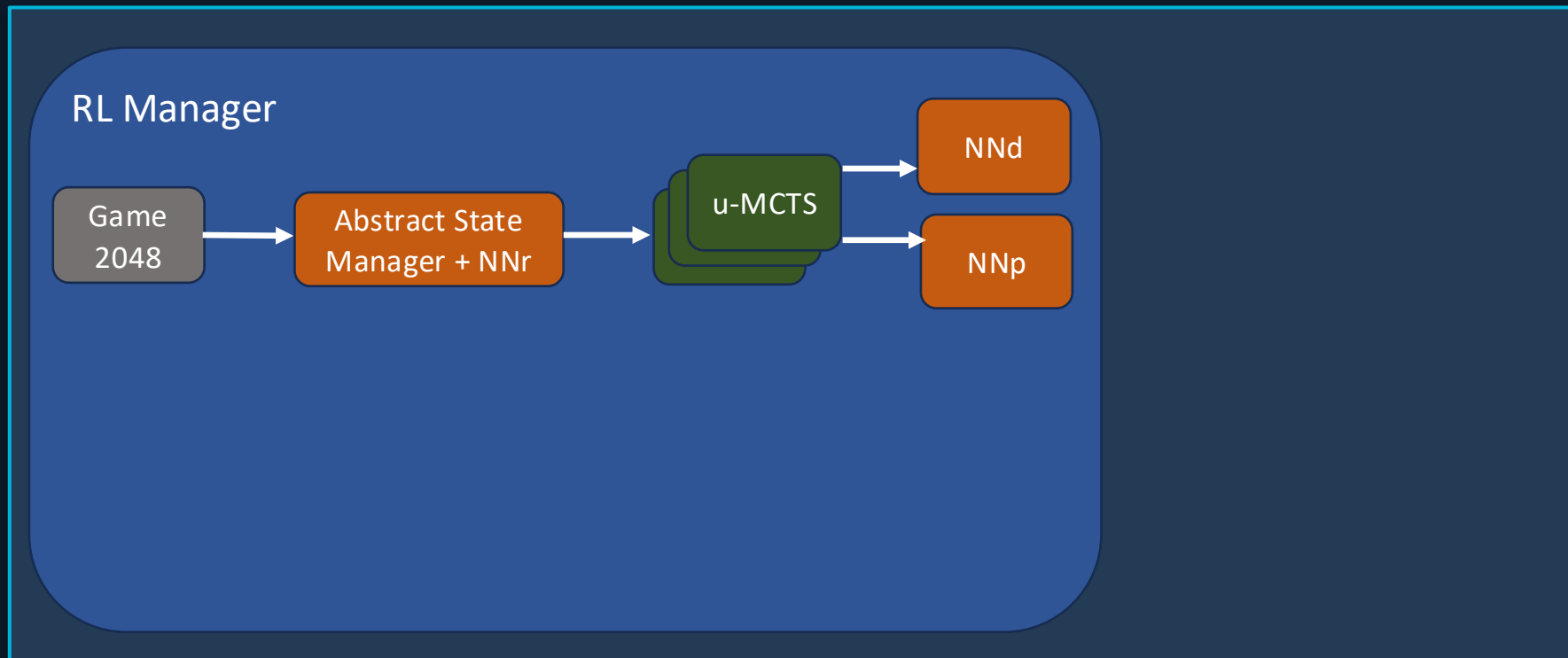


MuZero Implementation

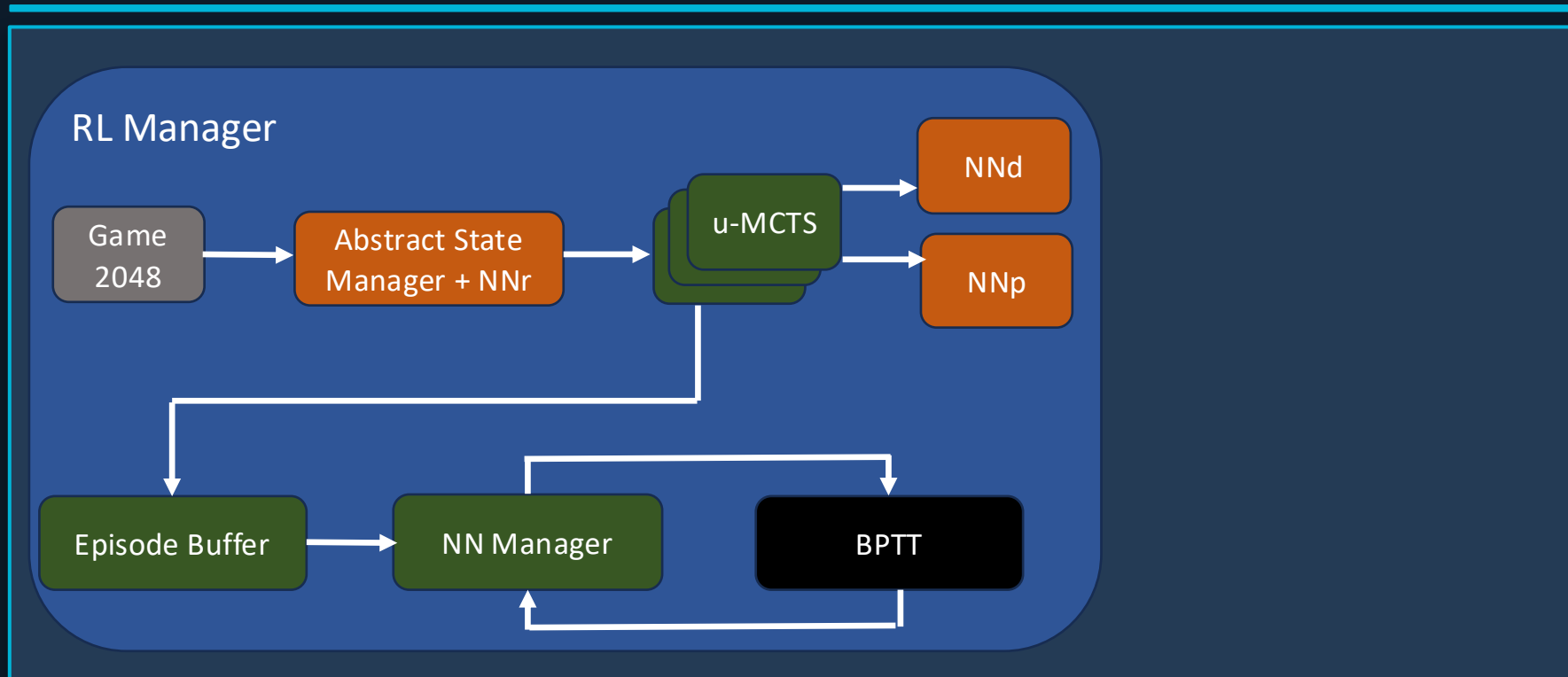
RL Manager

The diagram consists of a light blue outer frame. Inside this frame is a dark blue rectangular area. Within the dark blue area is a large, rounded blue rectangle. The text 'RL Manager' is positioned in the top-left corner of this rounded rectangle.

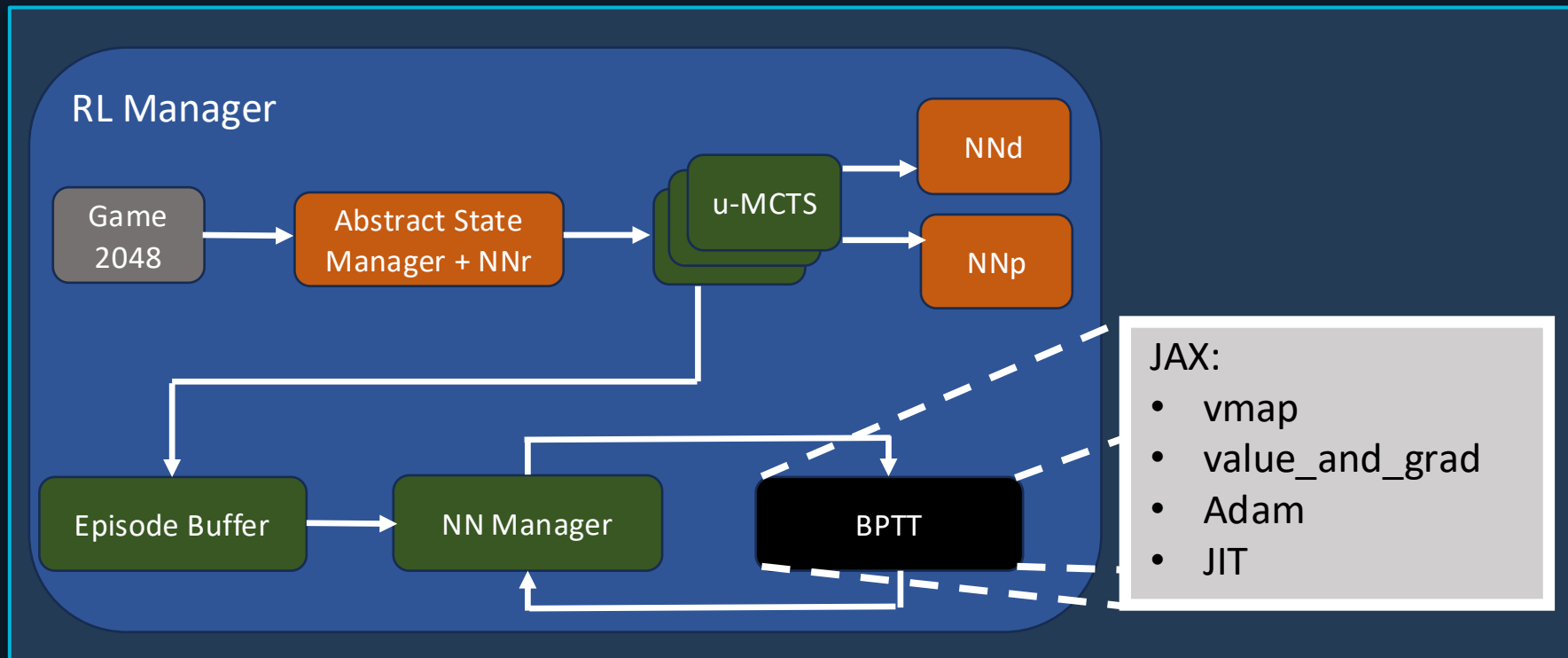
MuZero Implementation



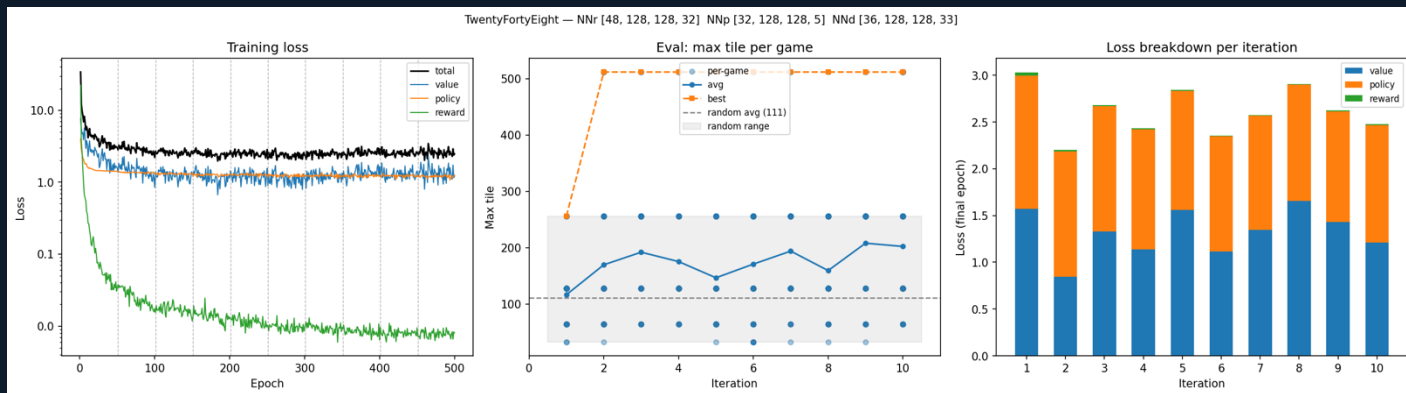
MuZero Implementation



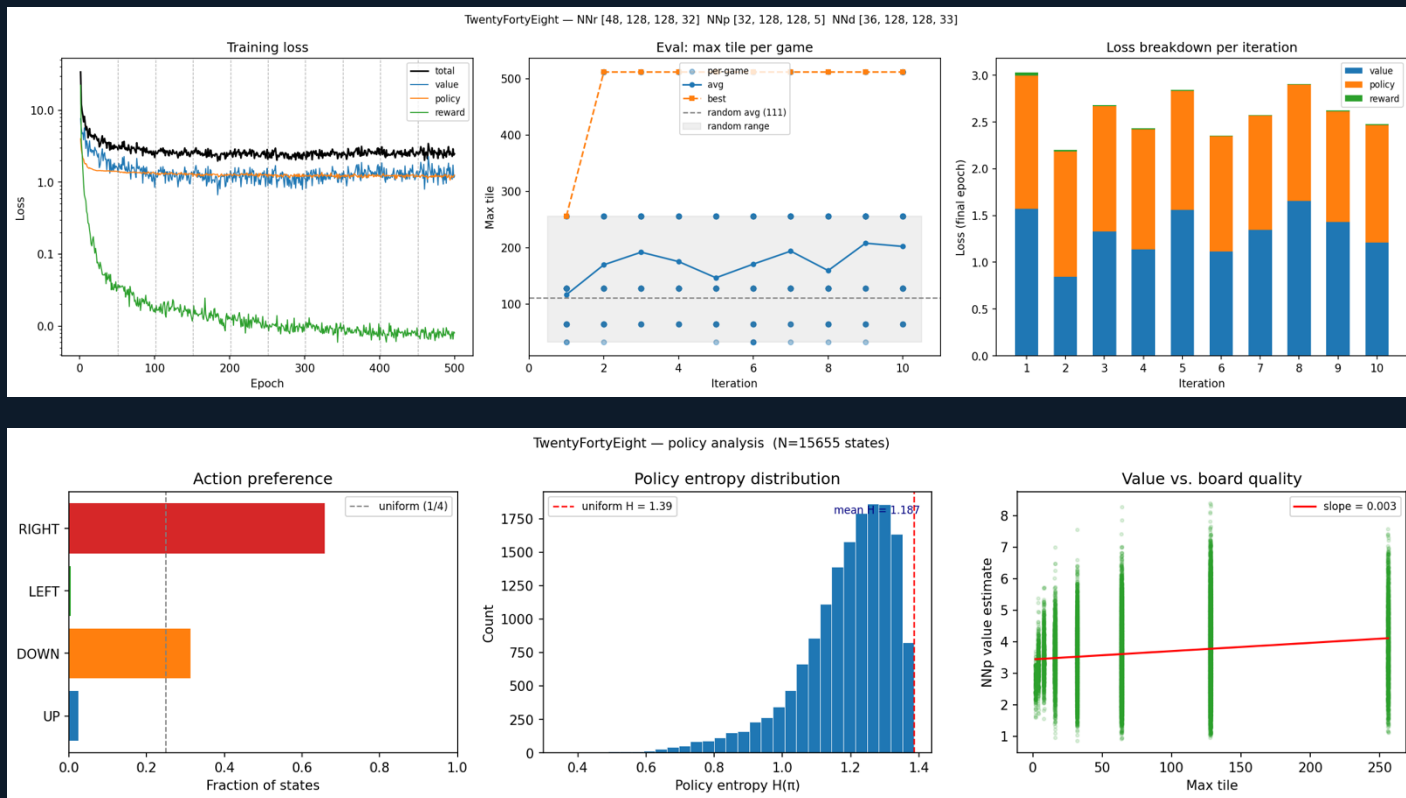
MuZero Implementation



Results on 2048



Results on 2048



Credits

Jørgen Hauan Larsen

- MuZero implementation
- Video script & presentation

Liam Robert Shepard

- MuZero implementation
- Video script & presentation
- Voice over

Torstein Hauge Fossnes

- MuZero implementation
- Video script & presentation

All group members independently implemented MuZero in parallel.

The implementation presented here is based on Jørgen Hauan Larsen's version.

Game visualized using matplotlib

Credits

Jørgen Hauan Larsen

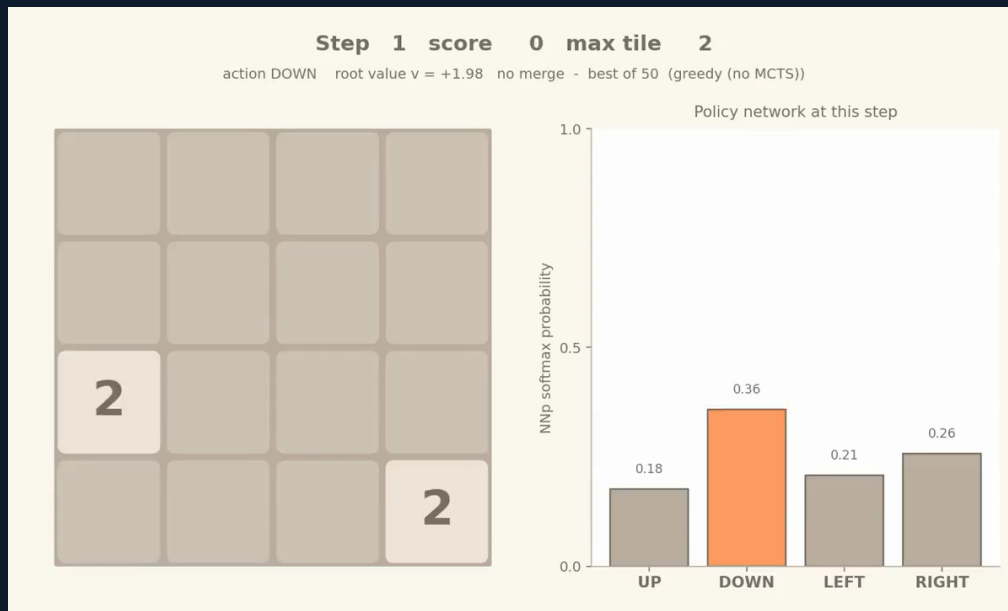
- MuZero implementation
- Video script & presentation

Liam Robert Shepard

- MuZero implementation
- Video script & presentation
- Voice over

Torstein Hauge Fossnes

- MuZero implementation
- Video script & presentation



All group members independently implemented MuZero in parallel.

The implementation presented here is based on Jørgen Hauan Larsen's version.

Game visualized using matplotlib